

Task 1:

To Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output and Design Parser Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands, Produce Execution Structures

```
suresh@DESKTOP-QCI8C9M: ~  
suresh@DESKTOP-QCI8C9M:~$ nano waitpid_demo.c  
suresh@DESKTOP-QCI8C9M:~$ gcc waitpid_demo.c -o waitpid_demo  
suresh@DESKTOP-QCI8C9M:~$ ./waitpid_demo  
Parent process started. PID = 780  
Parent is waiting for child PID = 781  
Child process started. PID = 781  
Child process finished.  
Child terminated normally.  
Child exit status = 10  
Parent process finished.  
suresh@DESKTOP-QCI8C9M:~$
```

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#include <ctype.h>
```

```
#define MAX_INPUT 1024
```

```
#define MAX_TOKENS 100
```

```
#define MAX_TOKEN_LEN 100
```

```

// Token structure
typedef struct {
    char value[MAX_TOKEN_LEN];
} Token;

// Split input into tokens
int tokenize(char *input, Token tokens[])
{
    int i = 0;
    int count = 0;

    while (input[i] != '\0' && count < MAX_TOKENS)
    {
        // Ignore whitespace
        while (isspace((unsigned char)input[i]))
        {
            i++;
        }

        if (input[i] == '\0')
            break;

        // Check delimiters
        if (input[i] == '|' || input[i] == ';')
        {
            tokens[count].value[0] = input[i];
            tokens[count].value[1] = '\0';
        }
    }
}

```

```

        count++;

        i++;
    }
    else
    {
        int j = 0;

        // Read normal token
        while (input[i] != '\0' &&
               !isspace((unsigned char)input[i]) &&
               input[i] != '|' &&
               input[i] != ';')
        {
            if (j < MAX_TOKEN_LEN - 1)
            {
                tokens[count].value[j] = input[i];

                j++;
            }

            i++;
        }

        tokens[count].value[j] = '\0';

        count++;
    }
}

return count;

```

```
}
```

```
// Display tokens for debugging
```

```
void display_tokens(Token tokens[], int count)
```

```
{
```

```
    printf("\n--- Token List ---\n");
```

```
    for (int i = 0; i < count; i++)
```

```
    {
```

```
        printf("Token %d: [%s]\n", i + 1, tokens[i].value);
```

```
    }
```

```
}
```

```
// Validate token stream
```

```
int validate_tokens(Token tokens[], int count)
```

```
{
```

```
    // Empty command
```

```
    if (count == 0)
```

```
    {
```

```
        printf("Error: Empty command.\n");
```

```
        return 0;
```

```
    }
```

```
// Command cannot start with |
```

```
if (strcmp(tokens[0].value, "|") == 0)
```

```
{
```

```
    printf("Error: Command cannot start with '|'.\n");
```

```
    return 0;
```

```

}

// Command cannot end with |
if (strcmp(tokens[count - 1].value, "|") == 0)
{
    printf("Error: Command cannot end with '|'.\n");
    return 0;
}

// Two pipes together are invalid
for (int i = 0; i < count - 1; i++)
{
    if (strcmp(tokens[i].value, "|") == 0 &&
        strcmp(tokens[i + 1].value, "|") == 0)
    {
        printf("Error: Invalid pipe sequence '| |'.\n");
        return 0;
    }
}

return 1;
}

// Generate execution structure
void create_execution_structure(Token tokens[], int count)
{
    printf("\n--- Execution Structure ---\n");

```

```

for (int i = 0; i < count; i++)
{
    if (strcmp(tokens[i].value, "|") == 0)
    {
        printf("PIPE\n");
    }
    else if (strcmp(tokens[i].value, ";") == 0)
    {
        printf("COMMAND SEPARATOR\n");
    }
    else
    {
        printf("COMMAND/ARGUMENT: %s\n", tokens[i].value);
    }
}
}

```

```

int main()
{
    char input[MAX_INPUT];
    Token tokens[MAX_TOKENS];

    printf("Enter command: ");

    // Read complete input
    if (fgets(input, sizeof(input), stdin) == NULL)
    {
        printf("Error reading input.\n");
    }
}

```

```
        return 1;
    }

    // Remove newline
    input[strcspn(input, "\n")] = '\0';

    // Tokenize input
    int count = tokenize(input, tokens);

    // Display tokens
    display_tokens(tokens, count);

    // Validate token stream
    if (validate_tokens(tokens, count))
    {
        printf("\nToken stream is valid.\n");

        // Create execution structure
        create_execution_structure(tokens, count);
    }

    return 0;
}
```

```
suresh@DESKTOP-QCI8C9M: ~  
suresh@DESKTOP-QCI8C9M:~$ nano T2.c  
suresh@DESKTOP-QCI8C9M:~$ gcc T2.c -o T2  
suresh@DESKTOP-QCI8C9M:~$ ./T2 pwd  
PATH = /usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/usr/lib/wsl/lib:/mnt/c/Program Files/WindowsApps/CanonicalGroupLimited.Ubuntu_2404.1.68.0_4_79rhkp1fndgsc:/mnt/c/Program Files/Common Files/Oracle/Java/javapath:/mnt/c/Program Files (x86)/Common Files/Oracle/Java/java8path:/mnt/c/Program Files (x86)/Common Files/Oracle/Java/javapath:/mnt/c/Windows/system32:/mnt/c/Windows:/mnt/c/Windows/System32/Wbem:/mnt/c/Windows/System32/WindowsPowerShell/v1.0:/mnt/c/Windows/System32/OpenSSH:/mnt/c/Users/Lenovo/AppData/Local/Programs/Python/Launcher:/mnt/c/Users/Lenovo/AppData/Local/Microsoft/WindowsApps:/mnt/d/c/sog/s 10 c/bin:/mnt/c/Program Files/Java/jdk-25:/mnt/c/Users/Lenovo/AppData/Local/Programs/Microsoft VS Code/bin:/mnt/c/Users/Lenovo/AppData/Local/Kingsoft/WPS Office/12.1.0.27458/clitools/snap/bin  
  
file found: /usr/bin/pwd  
execute permission: YES  
command resolved successfully.  
suresh@DESKTOP-QCI8C9M:~$ ./T2 Raghu  
PATH = /usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/usr/lib/wsl/lib:/mnt/c/Program Files/WindowsApps/CanonicalGroupLimited.Ubuntu_2404.1.68.0_4_79rhkp1fndgsc:/mnt/c/Program Files/Common Files/Oracle/Java/javapath:/mnt/c/Program Files (x86)/Common Files/Oracle/Java/java8path:/mnt/c/Program Files (x86)/Common Files/Oracle/Java/javapath:/mnt/c/Windows/system32:/mnt/c/Windows:/mnt/c/Windows/System32/Wbem:/mnt/c/Windows/System32/WindowsPowerShell/v1.0:/mnt/c/Windows/System32/OpenSSH:/mnt/c/Users/Lenovo/AppData/Local/Programs/Python/Launcher:/mnt/c/Users/Lenovo/AppData/Local/Microsoft/WindowsApps:/mnt/d/c/sog/s 10 c/bin:/mnt/c/Program Files/Java/jdk-25:/mnt/c/Users/Lenovo/AppData/Local/Programs/Microsoft VS Code/bin:/mnt/c/Users/Lenovo/AppData/Local/Kingsoft/WPS Office/12.1.0.27458/clitools/snap/bin  
  
command 'Raghu' not found in PATH.  
suresh@DESKTOP-QCI8C9M:~$
```

#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <ctype.h>

#include <unistd.h>

#include <sys/wait.h>

```

#define MAX_INPUT 1024

#define MAX_ARGS 100

#define MAX_ARG_LEN 256

/* Expand environment variables such as $USER */
void expand_variables(char *input, char *output)
{
    int i = 0, j = 0;

    while (input[i] != '\0' && j < MAX_ARG_LEN - 1)
    {
        if (input[i] == '$')
        {
            char variable[100];
            int k = 0;

            i++;

            while (isalnum((unsigned char)input[i]) ||
                input[i] == '_' )
            {
                if (k < 99)
                    variable[k++] = input[i];

                i++;
            }

```

```

variable[k] = '\0';

char *value = getenv(variable);

if (value != NULL)
{
    for (int x = 0; value[x] != '\0'; x++)
    {
        if (j < MAX_ARG_LEN - 1)
            output[j++] = value[x];
    }
}
else
{
    output[j++] = input[i++];
}
}

output[j] = '\0';
}

/* Parse input */
int parse_command(char *input, char *args[])
{
    int i = 0;

    int argc = 0;

```

```

while (input[i] != '\0' && argc < MAX_ARGS - 1)
{
    while (isspace((unsigned char)input[i]))
        i++;

    if (input[i] == '\0')
        break;

    char token[MAX_ARG_LEN];
    int j = 0;

    /* Double quoted string */
    if (input[i] == '"')
    {
        i++; // Skip opening quote

        while (input[i] != '\0' && input[i] != '"')
        {
            if (j < MAX_ARG_LEN - 1)
                token[j++] = input[i];

            i++;
        }

        /* No closing quote */
        if (input[i] != '"')
        {
            printf("\nSyntax Error: Unmatched double quote.\n");

```

```

        return -1;
    }

    i++; // Skip closing quote
}
else
{
    /* Normal token */
    while (input[i] != '\0' &&
           !isspace((unsigned char)input[i]))
    {
        if (j < MAX_ARG_LEN - 1)
            token[j++] = input[i];

        i++;
    }
}

token[j] = '\0';

/* Expand variables */
char expanded[MAX_ARG_LEN];
expand_variables(token, expanded);

args[argc] = malloc(strlen(expanded) + 1);

if (args[argc] == NULL)
{

```

```
    perror("malloc");  
    return -1;  
}
```

```
    strcpy(args[argc], expanded);  
    argc++;  
}
```

```
args[argc] = NULL;
```

```
return argc;  
}
```

```
int main()  
{  
    char input[MAX_INPUT];  
    char *args[MAX_ARGS];  
  
    printf("myShell> ");  
  
    if (fgets(input, sizeof(input), stdin) == NULL)  
        return 1;  
  
    input[strcspn(input, "\n")] = '\0';  
  
    /* Parse command */  
    int argc = parse_command(input, args);
```

```
if (argc < 0)
    return 1;

if (argc == 0)
{
    printf("Syntax Error: Empty command.\n");
    return 1;
}

/* Display parsed arguments */
printf("\n--- Parsed Arguments ---\n");

for (int i = 0; i < argc; i++)
{
    printf("Argument %d: %s\n", i + 1, args[i]);
}

/* Create child process */
pid_t pid = fork();

if (pid < 0)
{
    perror("fork");
    return 1;
}

if (pid == 0)
{

```

```

    printf("\nChild Process\n");
    printf("PID = %d\n", getpid());

    /* Execute command */
    execvp(args[0], args);

    perror("execvp");
    exit(1);
}
else
{
    wait(NULL);

    printf("\nChild process completed.\n");
}

/* Free memory */
for (int i = 0; i < argc; i++)
    free(args[i]);

return 0;
}

```

These two cases demonstrate both **successful command parsing/execution** and **syntax-error detection for unmatched quotes**.

